

Manual de operación

Cómo llevar el sistema sin ayuda: trabajar en tu ordenador, subir cambios a producción y salir del paso cuando algo falla. Escrito para leerlo con el teclado delante.



Qué hay en este manual

Si solo vas a leer un apartado, que sea el 3: es el que usarás cada vez que quieras que un cambio llegue a la web. El 4 se lee el día que algo se tuerce, y conviene haberlo ojeado antes.

01 El mapa

Las tres piezas y por dónde viaja el código

02 Trabajar en tu ordenador

Arrancar el sistema, ver los cambios y comprobarlos

03 Desplegar

De tu ordenador a tsports.tech, paso a paso

04 Cuando algo sale mal

Levantar el sitio, leer los registros y volver atrás

05 Administración

Cuentas, contraseñas y copias de la base de datos

06 Chuleta

Todos los comandos, en una sola página

El mapa

Hay tres piezas y el código pasa por las tres **siempre en el mismo orden**. Entender esto de una vez ahorra la mayoría de los sustos.

● Tu ordenador

Donde se escribe y se prueba. Nada de lo que hagas aquí se ve en internet hasta que lo subas.

● GitHub

Donde vive el código de verdad. Es el respaldo y la historia completa del proyecto.

● El servidor

El VPS que sirve tsports.tech. Se baja el código de GitHub, lo construye y lo publica.

● La base de datos

Vive dentro del servidor. No se sube nunca: las marcas y las cuentas solo existen ahí.

El camino del código

Es de un solo sentido y no tiene atajos:

- 1 Escribes en tu ordenador
- 2 Lo subes a GitHub con `git push`
- 3 Entrás al servidor y ejecutas el guion de despliegue

El guion se baja de GitHub lo que acabas de subir, lo construye y reinicia el sitio.

POR QUÉ ASÍ Y NO DIRECTO AL SERVIDOR

Antes se podía empujar el código al servidor sin pasar por GitHub, y eso permitía que el sitio estuviera corriendo una versión que no estaba guardada en ningún sitio. Si el VPS se perdía, se perdía con él. Ahora el servidor solo sabe bajar de GitHub: es imposible tener publicado algo que no esté respaldado.

Datos del servidor

QUÉ	VALOR
Dirección	2.25.152.122 , usuario root
Llave de acceso	~/.ssh/tsports_vps en tu ordenador
Carpeta del proyecto	/var/www/tsports
Sitio	https://tsports.tech
Repositorio	github.com/ILinZ/tsports , rama main
Base de datos	MariaDB, base tsports
Lo instalado	PHP 8.4 · Node 22 · nginx 1.26

LA LLAVE NO SE COMPARTE

El fichero ~/.ssh/tsports_vps es lo único que deja entrar al servidor. No se manda por WhatsApp ni se sube al repositorio. Si se pierde, se genera otra y se cambia en el servidor; si se filtra, cualquiera es administrador de la máquina.

Trabajar en tu ordenador

El sistema son dos programas que se arrancan por separado y se hablan entre sí: el servidor de datos y la interfaz.

Arrancarlo

Dos terminales, una para cada uno. Los dos se quedan corriendo: no se cierran hasta que termines.

1 El servidor de datos

`cd backend` y luego `php artisan serve`. Queda escuchando en `127.0.0.1:8000`.

2 La interfaz

`cd frontend` y luego `npm run dev`. Abre `http://localhost:5173` en el navegador.

Se entra con las mismas cuentas de siempre. La interfaz manda las peticiones al `8000` por su cuenta; no hay que configurar nada.

SI VES OTRA WEB QUE NO ES LA TUYA

Tienes otro proyecto en el ordenador, `linzdev`, que usa esos mismos dos puertos. Si estaba arrancado, se los queda él y tú acabas mirando su web sin que nada dé error. Ciérralo antes, o comprueba quién tiene el puerto.

Antes de dar nada por terminado

Dos comprobaciones. Las dos son rápidas y las dos evitan subir algo roto:

1 Que la interfaz compila

`cd frontend` y `npm run build`. Incluye la comprobación de tipos: si no compila aquí, tampoco compilará en el servidor y el despliegue se detendrá.

2 Que las reglas del negocio siguen en pie

`cd backend` y `php artisan test`. Son las reglas escritas como pruebas: quién puede editar qué, cuándo cuenta el dinero, qué ve cada rol. Corren sobre una base de datos de mentira y no tocan la tuya.

HA SALIDO BIEN SI

El `build` termina diciendo `built in` y un tiempo, y las pruebas terminan en verde con una línea del estilo `Tests: 111 passed`. Si alguna sale roja, el nombre de la prueba dice qué regla se ha roto.

Desplegar

Llevar un cambio de tu ordenador a tsports.tech. Son seis pasos y siempre los mismos. Los tres primeros en tu ordenador, los tres últimos en el servidor.

En tu ordenador

1 Comprueba que compila y que pasa

`cd frontend` y `npm run build`; luego `cd backend` y `php artisan test`. Si algo falla, para aquí: no subas nada.

2 Guarda los cambios

`git add -A` y luego `git commit -m "Lo que hiciste, en una frase"`.

3 Súbelos a GitHub

`git push origin main`

Esto todavía no cambia nada en la web. Solo deja el código guardado y listo para que el servidor lo recoja.

En el servidor

1 Entra por SSH

```
ssh -i ~/.ssh/tsports_vps root@2.25.152.122
```

2 Ejecuta el guion de despliegue

`cd /var/www/tsports` y luego `./deploy/desplegar.sh`

Tarda un par de minutos y va contando lo que hace. No lo interrumpas.

3 Comprueba que el sitio responde

`curl -s https://tsports.tech/up`, o simplemente abre la web en el navegador y entra.

HA SALIDO BIEN SI

El guion termina con **Despliegue terminado** en verde, y la web carga y te deja entrar. Si además cambiaste algo visible, recarga con Ctrl+F5 la primera vez: el navegador guarda la versión anterior.

Qué hace el guion mientras esperas

Para que sepas qué estás viendo pasar, y en qué punto se ha quedado si se detiene:

PASO	QUÉ SIGNIFICA
Modo mantenimiento	La web deja de atender un momento, para que nadie escriba mientras cambia la base de datos.
Bajando los cambios	Se trae de GitHub lo que subiste.
Dependencias de PHP	Instala las librerías del servidor.
Migraciones	Aplica los cambios de estructura a la base de datos.
Dependencias de Node	Instala las librerías de la interfaz.
Construyendo el frontend	Genera la web final. Aquí es donde falla si el código no compila.
Reiniciando	Recarga PHP y nginx para que sirvan lo nuevo.
Saliendo de mantenimiento	La web vuelve a atender.

SI EL GUION SE DETIENE, EL SITIO NO SE CAE

Mientras la construcción no termine bien, sigue publicándose la versión anterior. Y pase lo que pase, el guion saca la web del modo mantenimiento antes de salir. Un despliegue fallido deja el sitio como estaba, no roto.

Si el cambio toca la base de datos

Cuando el despliegue vaya a aplicar migraciones que borren o transformen datos, haz una copia antes. Está en el apartado 5, y son diez segundos.

Cuando algo sale mal

Por orden: primero mira qué pasa, luego decide. Casi nada de lo que puede fallar es irreversible.

La web dice que está en mantenimiento y no vuelve

Pasa si el despliegue se cortó a la mitad. Se saca a mano:

- 1 Entra al servidor y ve a la carpeta del proyecto

```
cd /var/www/tsports
```

- 2 Levántala

```
php backend/artisan up
```

Ver qué ha fallado

DÓNDE MIRAR	QUÉ CUENTA
<code>backend/storage/logs/</code>	Los errores de la aplicación, un fichero por día. El de hoy es el que lleva la fecha de hoy.
<code>/var/log/nginx/error.log</code>	Cuando la web ni siquiera carga.
<code>curl -s https://tsports.tech/up</code>	Si responde, la aplicación está viva.

Para ver el final de un registro sin abrirlo entero: `tail -50 backend/storage/logs/laravel-2026-09-04.log`

Volver a la versión anterior

Si el cambio que acabas de subir rompió algo y no ves el arreglo rápido, se vuelve atrás. En tu ordenador, no en el servidor:

- 1 Mira los últimos cambios

```
git log --oneline -5
```

- 2 Deshaz el último creando otro que lo revierte

```
git revert HEAD
```

 . Se abre un editor: guarda y cierra.

- 3 Súbelo y vuelve a desplegar

```
git push origin main
```

 , y en el servidor otra vez `./deploy/desplegar.sh` .

REVERT, NO RESET

`git revert` añade un cambio nuevo que deshace el anterior y deja el historial intacto. `git reset --hard` borra historia, y si ya estaba subida a GitHub, el siguiente `push` te va a pelear. Usa siempre revert.

VOLVER ATRÁS NO DESHACE LA BASE DE DATOS

Si el despliegue aplicó una migración que borró una columna, revertir el código no devuelve los datos: eso solo lo hace restaurar una copia. Por eso la copia se hace **antes**.

Administración

Lo que se hace de vez en cuando. Casi todo desde la propia web; solo las copias de la base piden terminal.

Cuentas y contraseñas

Todo esto se hace desde **Equipo** en el panel, siendo administrador. No hace falta tocar el servidor:

- **Dar de alta a alguien.** Botón Nueva cuenta. La contraseña la escribes tú y se la entregas; la persona la cambia luego desde su perfil.
- **Cambiar una contraseña olvidada.** Se edita la cuenta y se escribe una nueva. No hay correo de recuperación.
- **Quitar el acceso a alguien.** Se desactiva la cuenta, no se borra: así su rastro en el historial sigue teniendo nombre.

PENDIENTE: SIETE CUENTAS CON LA CONTRASEÑA PUBLICADA

La contraseña temporal con la que quedaron las cuentas importadas estuvo dentro del repositorio público, y siete cuentas de producción todavía entran con ella. Entre ellas hay un administrador.

Mientras no se cambien una por una desde **Equipo**, cualquiera que encuentre el repositorio puede entrar como administrador. Es lo más urgente de esta lista.

Copia de seguridad de la base de datos

No hay ninguna automática: se hacen a mano. Hazla antes de cada despliegue que toque datos, y de vez en cuando porque sí.

1 Entra al servidor y ve al backend

```
cd /var/www/tsports/backend
```

2 Lanza la copia

Las credenciales se leen del `.env`, así que no hay que escribir ninguna contraseña en el comando:

```
MYSQL_PWD=$(grep -m1 DB_PASSWORD= .env | cut -d= -f2-) mysqldump -u $(grep -m1 DB_USERNAME= .env | cut -d= -f2-) tsports | gzip > /var/backups/tsports/copia-$(date +%F-%H%M).sql.gz
```

3 Comprueba que no salió vacía

```
ls -lh /var/backups/tsports/
```

. Una copia buena pesa decenas de kilobytes, no cero.

UNA COPIA DENTRO DEL MISMO SERVIDOR NO ES UNA COPIA

Si el VPS se pierde, se pierden con él. Bájalas de vez en cuando a tu ordenador:

```
scp -i ~/.ssh/tsports_vps root@2.25.152.122:/var/backups/tsports/*.gz .
```

Restaurar una copia

Se hace muy de tarde en tarde y machaca lo que haya. Con calma:

1 Haz una copia de lo que hay ahora

Aunque esté mal. Es la red de seguridad por si la restauración tampoco era la que buscabas.

2 Restaura

Con los mismos datos que en el paso anterior:

```
zcat LA-COPIA.sql.gz | MYSQL_PWD=... mysql -u ... tsports
```

3 Limpia la caché de la aplicación

`php artisan optimize:clear` , desde `/var/www/tsports/backend` .

Chuleta

Todo lo del manual, sin explicaciones. Esta es la página para imprimir y dejar al lado del teclado.

Desplegar un cambio

COMANDO	DÓNDE
<code>npm run build</code> (en <code>frontend</code>)	Tu PC
<code>php artisan test</code> (en <code>backend</code>)	Tu PC
<code>git add -A</code>	Tu PC
<code>git commit -m "..."</code>	Tu PC
<code>git push origin main</code>	Tu PC
<code>ssh -i ~/.ssh/tsports_vps root@2.25.152.122</code>	Tu PC
<code>cd /var/www/tsports</code>	Servidor
<code>./deploy/desplegar.sh</code>	Servidor
<code>curl -s https://tsports.tech/up</code>	Servidor

Trabajar en local

COMANDO	PARA QUÉ
<code>php artisan serve</code> (en <code>backend</code>)	Servidor de datos, puerto 8000
<code>npm run dev</code> (en <code>frontend</code>)	Interfaz, puerto 5173
<code>php artisan test</code>	Las reglas del negocio
<code>npm run build</code>	Compilar y comprobar tipos

Cuando algo falla

COMANDO	PARA QUÉ
<code>php backend/artisan up</code>	Sacar la web de mantenimiento
<code>tail -50 backend/storage/logs/laravel-FECHA.log</code>	Ver los últimos errores
<code>git log --oneline -5</code>	Ver los últimos cambios
<code>git revert HEAD</code>	Deshacer el último cambio
<code>git status</code>	Ver qué tienes sin guardar

LA REGLA DE ORO

El código va siempre en el mismo sentido: tu ordenador, GitHub, el servidor. Si alguna vez te ves editando ficheros directamente en el servidor, para: ese cambio se perderá en el siguiente despliegue y no estará en ningún respaldo.